

Dispositivos Conectados

Interfaces Gráficas: TFT Adafruit 0.96" 160x80

2025/2026

Authors: Paulo C. Bartolomeu

1 Objetivos

- Familiarização com o *driver* SS7735S do *display* TFT Adafruit 0.96"
- Familiarização com uma biblioteca gráfica básica para desenho de formas geométricas, linhas e texto
- Construção de interfaces gráficas *multi-screen* para dispositivos IoT

2 Introdução

No contexto da Internet das Coisas, é cada vez mais importante que os dispositivos permitam uma interação clara e eficiente com os utilizadores. As Interfaces Homem-Máquina (IHM) são fundamentais para tornar a informação gerada por sistemas inteligentes mais acessível e compreensível.

Este guião tem como objetivo demonstrar essa interação através da utilização do ESP32-C6 com um ecrã TFT de 0,96 polegadas da Adafruit. Através desta combinação, pretende-se criar uma interface visual simples e funcional, capaz de apresentar informação em tempo-real, melhorando a interação entre o utilizador e o dispositivo e potenciando um acesso fácil ao contexto do dispositivo.

3 Trabalho Prático

O trabalho prático desta aula desenvolve-se com base no *driver* e biblioteca gráfica disponíveis no repositório [ST7735-ESP32-C6](#). Adicionalmente, **deverá ser consultada a ficha técnica do *display* Adafruit 160x80 0.96"** disponível na página do *elearning* na pasta Recursos.

O repositório [ST7735-ESP32-C6](#) oferece uma base modular e eficiente para comunicar com o ecrã TFT através da interface SPI, incluindo funções para inicialização, controlo de cores, desenho de formas básicas, apresentação de texto e atualização do conteúdo do ecrã. A estrutura do código permite ainda uma fácil extensão, facilitando a criação de menus interativos, gráficos de dados em tempo-real ou indicadores de estado de sensores, todos elementos típicos de uma IHM funcional.

3.1 Funcionamento elementar

O primeiro passo deste guião consiste em montar o *display* TFT na placa brance e ligá-lo ao ESP32-C6. Para o efeito, use o circuito da Figura 1. Note que o pino de *brightness* (LIT) está ligado a um GPIO genérico dos ESP32. O *chip select* para o leitor de cartões SD está ligado a 3.3V de modo a assegurar que a interação via SPI ocorre exclusivamente com o TFT.

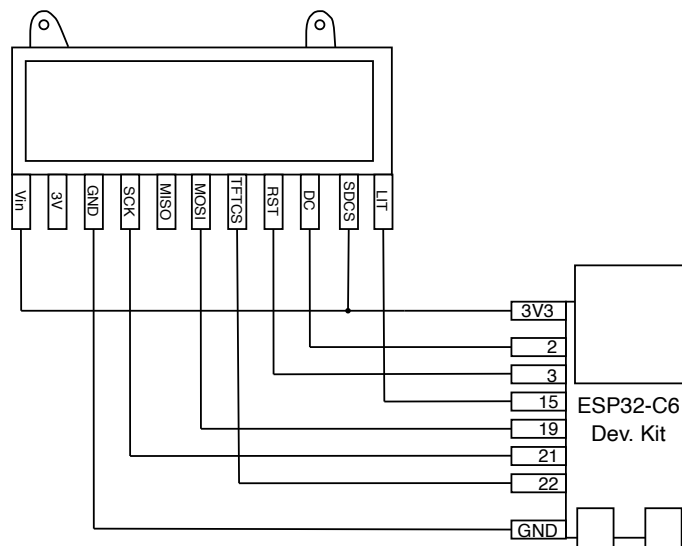


Figura 1: Esquema de ligação do *display* TFT Adafruit 0.96"

Nota Importante! Por lapso, o guião da aula 5 representava um circuito para ligação do leitor de cartões SD com resistências de *pull-up*, tratando-se isso era uma reminiscência do circuito anterior dos sensores TC74 e DHT20 que empregam o protocolo I2C e que, **efetivamente**, carecem de resistências de *pull-up*. O guião da aula 5 já foi atualizado removendo estas resistências desnecessárias. Por isso, o circuito apresentado neste guião não as inclui.

Depois de efetuar as ligações indicadas, é necessário descarregar a aplicação de exemplo que será usada neste guião. Para o efeito execute o seguinte comando no seu terminal. Note que tem de previamente ter instalado a aplicação *git* no seu sistema.

```
~$ git clone https://github.com/T4V4RES/Driver-ST7735-ESP32-C6.git
```

Abra o projecto no *Visual Studio Code* e altere o Device Target de esp32 para esp32c6.

QUESTÃO 1: Analise o repositório, estudando o ficheiro README, o driver e a biblioteca gráfica. Familiarize-se também com o exemplo. Compile o programa e descarregue-o para a placa. Deverá observar no terminal o seguinte log de eventos. A sequência de operações visível no *display* TFT é a esperada?

```
...
I (256) MAIN_APP: =====
I (256) MAIN_APP: Iniciando Display ST7735S Demo
I (256) MAIN_APP: =====
I (266) ST7735: ST7735 Driver - Adafruit Mini TFT 0.96
I (266) ST7735: Pinos: MOSI=19 CLK=21 CS=22 DC=2 RST=3 BL=15
I (276) ST7735: Backlight ON
I (276) ST7735: SPI @ 8 MHz
I (1386) ST7735: Display OK: 160x80 pixels
I (1386) MAIN_APP: Display inicializado com sucesso!
I (1386) MAIN_APP: A testar cores...
I (6466) MAIN_APP: A desenhar linhas...
```

```
I (7486) MAIN_APP: A escrever texto...
I (8616) MAIN_APP: A desenhar linhas...
I (9636) MAIN_APP: A escrever texto...
```

QUESTÃO 2: Analise o ficheiro README e teste os exemplos 1, 2 e 3, aí indicados. Realize os testes com diferentes parametrizações de cor, tamanho de letra, tamanho e racio de retangulos e espessura de linha. Nesta pergunta deverá familiarizar-se com a implementação das diferentes funções da biblioteca gráfica.

QUESTÃO 3: Assumindo que se pretende emular o funcionamento de um LED RGB no *display* TFT para assinalar o *status* de uma operação de acordo com a seguinte definição, implemente um programa que apresente uma circunferência de raio fixo centrada no ecrã e que alterne entre as 3 cores possíveis uma vez por segundo.

```
{
{"ok", green},
{"error", red},
{"fault", yellow}
}
```

QUESTÃO 4: Desenvolva uma interface gráfica para apresentar informação de ID, temperatura, humidade e estado de operação de um dispositivo IoT. Considere que o estado apresentado pelo dispositivo obedece aos *thresholds*: {frio: < 22 graus}, {alerta: > 30 graus} e normal nos restantes casos. A interface gráfica a construir deve usar dados aleatórios de temperatura e humidade, atualizando a informação apresentada a cada segundo. Esta interface gráfica deve obedecer ao *layout* apresentado nas Figuras 2, 3 e 4.



Figura 2: *Layout* da IHM para o estado NORMAL

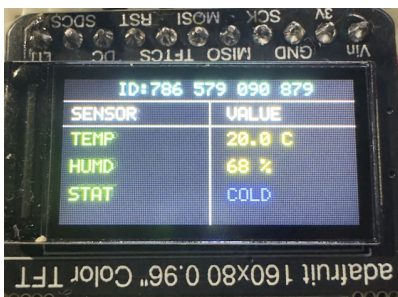


Figura 3: *Layout* da IHM para o estado COLD

QUESTÃO 5: Integre a interface desenvolvida com o sensor de temperatura e humidade

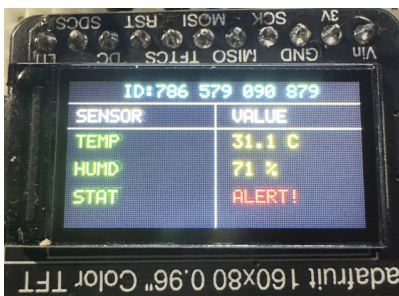


Figura 4: *Layout* da IHM para o estado **ALERT**

DHT20 usado nas aulas anteriores. Teste-o aquecendo o sensor e verificando que o estado do dispositivo se altera em conformidade.

QUESTÃO 6: Utilizando um botão de pressão, crie uma interface gráfica que faça o *toggle* entre dois *screens*. O *screen default* é o que desenvolveu na pergunta anterior. O *screen* alternativo deve mostrar as últimas mensagens de log do sistema, uma por linha (até cinco 5 linhas no total). Sempre que o utilizador alterna entre *screens* pressionando o botão, próximo deve ser apresentado. Nota: deve haver suporte para uma lista de mensagens superior a 5, observando-se a evolução simulada dos *logs* ao longo do tempo.

QUESTÃO 7: Na sequência da pergunta 6, crie um *screen* adicional onde se possa visualizar o gráfico da temperatura ao longo do tempo, em tempo-real. Os eixos do gráfico devem estar assinalados. Os pontos devem ser apresentados a partir da origem, ou seja, $t=0$ representa o instante em que o utilizador “entrou” no *screen*, sendo os valores de temperatura obtidos daí em diante representados no gráfico. A funcionalidade de rotação entre *screens* através do uso do botão de pressão deve manter-se operacional.

QUESTÃO 8: Evolua o *screen* anterior para representar também a humidade no mesmo gráfico, mas com uma cor diferente e um segundo eixo das ordenadas. As características funcionais são iguais às definidas para a questão 7.

QUESTÃO 9: Altere a interface de modo a que sejam representados nos gráficos de temperatura e humidade apenas os últimos x pontos que “caibam” no gráfico. Note que se o utilizador estiver a consultar o *screen default* ou o de *log*, os dados de temperatura e humidade deverão continuar a ser produzidos à mesma frequência e devem ser guardados. Quando regressar ao *screen* de visualização do gráfico, os x pontos mais recentes que foram registados devem ser apresentados, sendo o gráfico integralmente *redendered* a cada nova medição de temperatura e humidade.

4 Acknowledgements

Originally authored by [Paulo C. Bartolomeu](#)

Driver originally adapted for the ESP32-C6 SoC (from the Arduino’s Adafruit library) by [Guilherme Tavares](#)